

Comparative Analysis of Randomness in Pseudo-Random Number Generators

Mingheng Su, Zhuoran Lyu, Yulin Chen

Dr. Diana Skrzydło

STAT 430

July 2026

Contents

1	Introduction	3
2	Experiment	4
2.1	Design & Plan	4
2.1.1	Response Variables	4
2.1.2	Treatment Groups	4
2.1.3	Model	5
2.2	Data	7
2.3	Graphic Analysis	7
2.4	Analysis	7
2.4.1	ANOVA	7
2.4.2	Comparison Tests	8
2.4.3	Contrasts	9
2.4.4	Residual Analysis	10
3	Conclusion	12
A	Randomness Tests	13
A.1	Entropy	13
A.2	The Runs Test (for reference only)	13
A.3	The Distribution Uniformity Test	13
A.4	Autocorrelation	14
A.5	π -Test	15
A.6	Repetition Test	15
A.7	Spectral Periodicity (Not Used)	15
A.8	Overall Score	16
B	PRNG Algorithms	17
B.1	Linear Congruence Generator (LCG)	17
B.2	Mersenne Twister	18
B.3	Xor Shift	18
C	Other Fitted Models	19
C.1	Model Assuming Constant Variance	19
C.2	Model with Blocking Factor Seed	20
C.3	Linear Model	20
C.4	Data Transformations	21

D	R Code and Output	22
D.1	Graphics	22
D.1.1	Raw Data	22
D.1.2	Seed	24
D.1.3	Residuals	26
D.2	Model Fitting	29
D.2.1	Testing Equal Variances	29
D.2.2	Model	31
D.3	Comparison Tests and Contrasts	33
D.4	Residual Analysis	36
E	Application: The Monte Carlo Estimation	39
E.1	Logics	39
E.2	Estimating π	40

Chapter 1

Introduction

A sequence of integers is said to be random if they reveal no patterns or trends, and the future numbers are unpredictable given previous numbers. For example, if we randomly pick numbers between 1 and 100, we might get the sequence

1, 19, 88, 23, 42, 78, 4, ...

Ideally, the sequence should behave like an independent sample of discrete $U(0, 100)$ variables^{1.1}. Such random sequences are useful for many applications, such as the *Monte Carlo Estimator* (E). One way of obtaining random sequences is to sample from real world experiments, for example, toss a die multiple times and record the numbers facing up; this is called the truly random samples, but is very inefficient for long sequences. Luckily, modern computer algorithms can effectively generate *pseudo-random sequences*.

A *pseudo-random number generator* (PRNG) is a deterministic algorithm that recursively generates random numbers based on a seed, which is the first number to start with. We implemented four PRNGs for our experiment: the *Linear Congruence Generator* (LCG), the *RANDU* variant, the *Mersenne Twister* (MT) variant, and the *Xor Shift* (XOR) variant. For fairness, they are designed to produce sequences with integers chosen from $[1, 1024]$. An algorithm generates a unique sequence based on a specific seed, however, the seed should not affect the randomness for good algorithms. Such generated pseudo-random sequences are not necessarily random since they are predictable if we know the formula, but with a great algorithm, the numbers can exhibit features of fairly high randomness. We are interested in which PRNGs can produce the most random sequences.

The *length* of a sequence generated by a PRNG might also affect the randomness. Some algorithms' sequences reveal significant correlation between close or adjacent numbers but show no long-term relationship if the sequence is long enough; some algorithms such as the LCG has a period, after which the numbers start to repeat the previous pattern and imply poor randomness. Intuitively, a sequence generated by an algorithm should be poor for small and large lengths, and be better for moderate lengths. Therefore, we are also interested in whether the length has an effect on the randomness of those pseudo-random sequences.

For each sequence, we have a function to evaluate its *randomness score*, a normalized continuous index between 0 and 1. A sequence is considered fairly random if its randomness score is close to 1, and poor if the score is close to 0. Additionally, the seed is included as a blocking variable to eliminate the seed-related effects on randomness score. As stated before, a seed should be independent of the randomness score, but we want to account for potential seed-related flaws in our algorithms, such as the poor performance associated with large seeds. Nevertheless, whether to include or discard the seed factor is ultimately decided based on its statistical significance reflected by data.

Question: Holding the seed as a blocking variable, how do the proposed algorithms and sequence lengths affect the randomness of generated integer sequences in $[1, 1024]$, and which algorithm-length combinations achieve the highest randomness?

^{1.1}If divide every number by the possible maximum number, which is 100 in this example, the values should be uniformly distributed over finite grids in $[0, 1]$ (very close to the continuous $U(0, 1)$ with a sufficiently long sequence). This is called the "normalization". In our experiment, the maximum integer is 1024.

Chapter 2

Experiment

2.1 Design & Plan

2.1.1 Response Variables

For each sequence generated by an algorithm, its overall randomness score S is a continuous value within $[0, 1]$ reflecting the randomness. We conducted 6 separate tests which all provide a normalized score within $[0, 1]$ (similarly, higher the better) and the final score is chosen to be a weighted average of 5 of them (not including the runs test):

- Entropy: Evaluates the uncertainty level of the sequence;
- Runs Test (for reference only): Evaluates how random the runs of data are;
- Uniformity Test: Calculates the distributional variation distance between the sequence and the theoretical $U(0, 1)$;
- Average Autocorrelation: Calculates the average correlations over 200 lags;
- π -Test: Penalizes large discrepancy between the real and estimated π using the Monte Carlo estimator;
- Repetition Test: Penalizes highly repetitive patterns.

More details can be found in Appendix (A). Each individual test might not be enough for testing randomness; this is why we look at all of them. For example, entropy evaluates frequencies of integers but ignores repeated patterns that can only be checked by repetition test; π -test works well only when the sequence is likely uniform, hence it needs to be combined with the uniformity test. The runs test is a pass or fail standard at 0.05 significance level which would lead to discrete jumps in the overall score, so for the sake of smoothness we remove or allocate very low weight to it. Except for runs test, the tests are chosen to be equally weighted to avoid manual intervention that distorts the final score.

A random sequence is expected to have a high score, while the converse is not necessarily true. The overall score is more like a preference score, not a deterministic measurement like length, weight, or category that can be measured precisely. Therefore, we might expect minor unreasonable variability. Generally speaking, as the test thoroughly assesses multiple aspects of randomness, including uncertainty, distributional uniformity and repeated patterns, a high score indicates that the sequence is very likely to exhibit random behaviors. However, like stated before, the score should be interpreted as a relative indicator of randomness rather than a precise measurement.

2.1.2 Treatment Groups

Main Factor α : Algorithm There are 4 algorithms,

- LCG ($i = 1$): The Linear Congruence Generator, which recursively feeds the previous integer to a linear congruence system and obtain the next integer in the sequence. This is a small model designed on our own.

- RDU ($i = 2$): It is a variant of the IBM's RANDU algorithm [1], which is a powerful LCG.
- MT ($i = 3$): The Mersenne Twister [MT], which is a powerful PRNG of exceptionally long period. It is a mixture of various randomization techniques.
- XOR ($i = 4$): The Xor Shift method [3], which is a lightweight PRNG that recursively applies XOR and bit shifts to create the next number.

More details can be found in (B). They are all commonly recognized and widely used PRNGs, but in this report we are investigating their performance when limiting the numbers to be within $[1, 1024]$.

Main Factor β : Length The lengths are measured at 5 representatives,

$$L = 300 \ (j = 1), \quad L = 600 \ (j = 2), \quad L = 900 \ (j = 3), \quad L = 1200 \ (j = 4), \quad L = 1500 \ (j = 5)$$

Therefore, there are $4 \times 5 = 20$ combinations of (*algorithm*, *length*).

Blocking Factor δ : Seed Choice The possible seeds are $1, 2, \dots, 1000$. We separate them into 4 levels,

- low: $[1, 250]$ ($k = 1$)
- middle-low: $[251, 500]$ ($k = 2$)
- middle-high: $[501, 750]$ ($k = 3$)
- high: $[751, 1000]$ ($k = 4$)

If we include seed as a blocking factor, there should be $4 \times 5 \times 4 = 80$ combinations of (*algorithm*, *length*, *seed*).

Held-Constant Factors: the maximum integer in the sequences, number of replications for each group, test suit

Allowed-to-Vary Factors: time for computing sequences, computational costs, devices/machines used for calculating the sequences (or by hand), date and time of the experiment

2.1.3 Model

The analysis indicates that the seed factor is not significant, as we expected. To save space, the model with seed is put in (C.2). We used the ANOVA model

$$y_{ijk} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \epsilon_{ijk} \sim N(\mu_{ij}, \sigma_{ij}^2) \quad (2.1)$$

for algorithm $i = 1, 2, 3, 4$, length $j = 1, 2, 3, 4, 5$, and seed choice $k = 1, 2, 3, 4$, with constraints

$$\sum_{i=1}^4 \alpha_i = \sum_{j=1}^5 \beta_j = \sum_{i=1}^4 (\alpha\beta)_{ij} = \sum_{j=1}^5 (\alpha\beta)_{ij} = 0$$

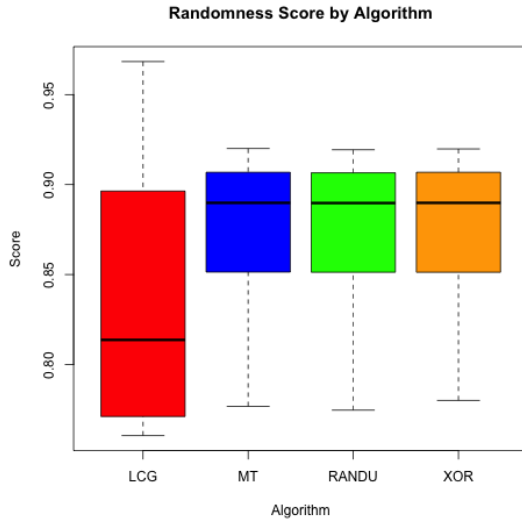
The parameters are:

- y_{ijk} : the randomness score of the k -th sequence generated by algorithm i at length j and with seed level k
- μ : grand mean of all data (1 df)
- α_i : effect of algorithm $i \in \{1, 2, 3, 4\}$ (3 df)
- β_j : effect of length $j \in \{1, 2, 3, 4, 5\}$ (4 df)

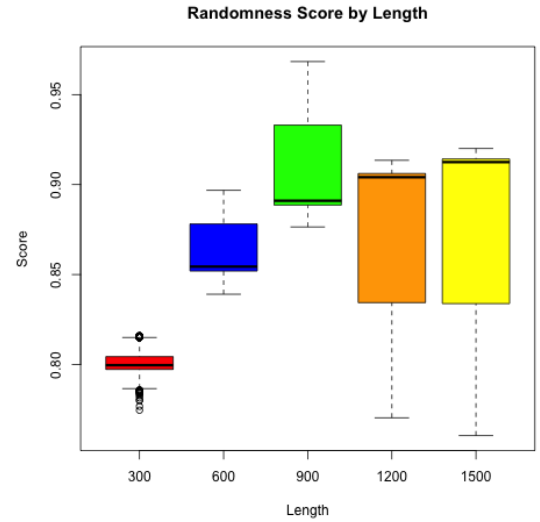
- $(\alpha\beta)_{ij}$: effect of algorithm i , length j interaction (12 df)
- $\sigma_{ij}^2 = \lambda_{ij}^2 \sigma^2$: variance of group (i, j)
- ϵ_{ijk} : iid $N(0, \sigma_{ij}^2)$ noise

where each σ_{ij}^2 defined by a scale of the baseline variance, σ^2 , which is the variance of group $(LCG, 300)$. λ_{ij}^2 is the scale factor in the variance of group (i, j) comparing to σ^2 (see D.2.3). The residuals have 19980 degrees of freedom in total.

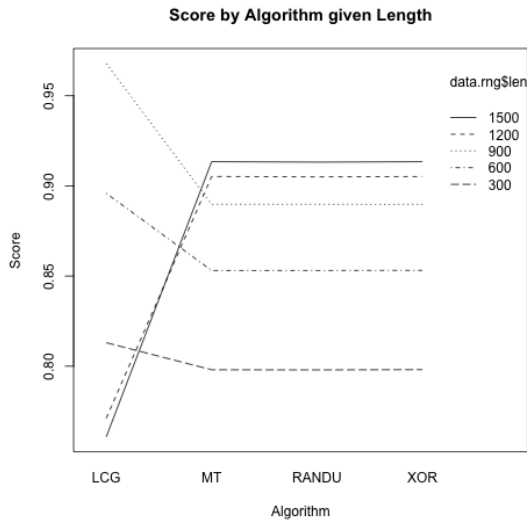
Figure 2.1.1: Plots of Main Factors



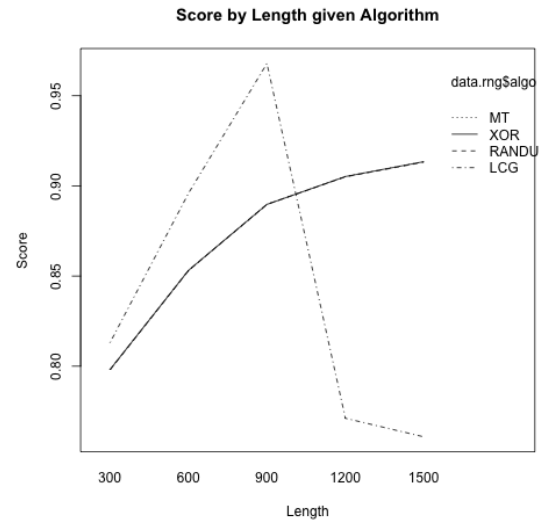
(a) Algorithm Box Plot



(b) Length Box Plot



(c) Interaction Plot of Algorithm given Length



(d) Interaction Plot of Length given Algorithm

2.2 Data

Our python programs and generated sequences can be found here [7]. Based on seeds from 1 to 1000, we constructed 1000 replications for each of the 20 (*algorithm, length*) combinations, so there are $20 \times 1000 = 20000$ data observations in total. To exemplify, for (*LCG*, 600), we apply LCG on seeds $1, 2, \dots, 1000$ to get 1000 sequences of length 600 and then evaluate their randomness scores as response variables; for (*XOR*, 1200), we apply XOR on seeds $1, 2, \dots, 1000$ to get 1000 sequences of length 1200 and then evaluate their randomness scores, etc. Table 2.2.1 summarizes the mean responses at each treatment combination.

	300	600	900	1200	1500
LCG	0.8130336	0.8958792	0.9679026	0.7711315	0.7608282
RDU	0.7979542	0.8530555	0.8897541	0.9050140	0.9131846
MT	0.7980290	0.8530525	0.8897689	0.9052040	0.9134406
XOR	0.7981164	0.8531312	0.8897224	0.9051761	0.9134276

Table 2.2.1: Mean Scores by Treatment Groups

2.3 Graphic Analysis

As we can see in Figure 2.1.1, both algorithm and length are effective since their different levels reveal different means. Refer to 2.1.1a. In general, the RANDU, MT and XOR methods seem to have incredibly similar performance and all greatly outperform the LCG method. With the large variability, the LCG method has less stable performance than the other three algorithms, but on the other hand, it also contains almost all top scores (93% – 100%) which the other three algorithms cannot achieve. From 2.1.1b we can see an increasing trend in the score as length grows from 300 to 900 but it drops after length 900. This is worth to be further investigated. Figure D.1.2a shows that most low-length groups (especially at length 300) and most LCG groups have massive left tails which mainly contribute to the left skewness. Additionally, some LCG groups have many gaps in the box plot, revealing that most of its scores lie in some evenly spaced discrete intervals. With these evidence, we might challenge the normality assumption on the observed data. However, other groups seem to be normal, though not perfect. The lines in the interaction plots 2.1.1c, 2.1.1d have many crossings and are not completely parallel, suggesting a potential interaction effect between algorithm and length.

Figure D.1.3 shows that the seed levels have very similar means. This suggests that seed has no significant effect on randomness score. From the interaction plots D.1.4, we see that both algorithm and length have no large interaction with seed since the lines are either parallel or coincident.

2.4 Analysis

2.4.1 ANOVA

We fitted model (C.2) (with seed) and model (2.1) (without seed). The ANOVA decomposition 2.4.1 indicates that seed has a p-value of 0.607(> .05), which is not significant. Nicely, algorithm, length and their interaction are significant since they have small p-values $2.2\text{e-}16$ (< .05). The Levene's test D.2.1 on the variance has p-value $2.2\text{e-}16$ (< .05) which suggests us to deal with unequal variance across treatment groups (see D.2.2, D.2.3). After fitting the two models while accounting for unequal variance, it turns out that they have similar performance shown in Table 2.4.1.

Model	R^2	AIC	BIC	Residual df	Residual Std.Err
With Seed	0.998	-200345.3	-200005.5	19977	0.002391629
No seed	0.998	-200413	-200096.9	19980	0.002391855

Table 2.4.1: Model Comparison

It seems that we don't necessarily need seed as a blocking factor. As well, the ANOVA test [D.2.1](#) on two models gives a p-value of 0.2039(> .05), so it is safe to exclude seed to avoid overfitting. Model [D.2.4](#) shows that all 20 parameters of the reduced model, including all main and interaction effects, are significant since they have small p-values (< .0001).

	Df	Sum Sq	Mean Sq	F value	Pr(>F)	
algo	3	3.40	1.134	1.983e+05	<2e-16	***
len	4	24.42	6.105	1.068e+06	<2e-16	***
seedLevel	3	0.00	0.000	6.130e-01	0.607	
algo:len	12	33.64	2.803	4.904e+05	<2e-16	***
Residuals	19977	0.11	0.000			

Figure 2.4.1: ANOVA Model with Seed

2.4.2 Comparison Tests

R code can be found in [D.3](#). We want to find the best (*algorithm, length*) combination that has the highest randomness score. Instead of constructing all possible pairwise contrasts which reduces statistical power, we identify the combination with the potential highest mean score and compare it with the other 19 combinations. In [2.2.1](#), (*LCG*, 900) has the highest mean score 96.79%. We set it as a reference and compare to all other groups. See [D.3.1](#), all subtractions are negative and have small p-values(< .0001), meaning that the differences are significant - (*LCG*, 900) is indeed the best combination.

Unlike other three algorithms that are professional models, the LCG method uses a simple model of small parameters that are specifically designed for generating numbers within [1, 1024]. We want to test whether LCG is really poorer than the other three algorithms. Define the contrasts at different lengths by

$$\hat{C}_{\text{ALGO}, L} = \bar{y}_{\text{ALGO}, L} - \bar{y}_{\text{LCG}, L} \quad (2.2)$$

where *ALGO* = *RDU*, *MT*, *XOR*, *L* = 300, 600, ..., 1500. In the Tukey comparisons of group means [2.4.2](#), all comparisons involving LCG are significant at all lengths. Interestingly, at lengths 300, 600, 900, LCG outperforms all three professional models as the differences are significantly positive and vary from 1.5% to 7.8% as length increases. Nevertheless, LCG is about 13% and 15% behind the other three algorithms at length 1200, 1500 respectively, revealing its weakness at long lengths.

We also want to see whether the professional models *RANDU*, *XOR*, and *MT* have the same performance. Define the three pairwise comparison contrasts by

$$\hat{C}_1 = \bar{y}_{\text{MT}} - \bar{y}_{\text{RDU}}, \quad \hat{C}_2 = \bar{y}_{\text{RDU}} - \bar{y}_{\text{XOR}}, \quad \hat{C}_3 = \bar{y}_{\text{MT}} - \bar{y}_{\text{XOR}} \quad (2.3)$$

for each length. In [2.4.2](#), only *MT-RANDU*, *RANDU-XOR* at length 1500 are significant since their p-values are less than 0.05. However, such small differences (around 0.03% and 0.02%) wouldn't matter a lot, especially when their means are high (all around 91.3%). We can deduce that the professional algorithms don't have a major difference at these lengths.

Thinking & Comments Note that 300, 600, 900 are less than LCG's period 1024 (see [B.1](#)) and seem better when closer to 1024. The length comparisons in [D.3.1](#) also prove that, when the length is less than 1024, a longer LCG yields a better score since the length comparisons 300-600, 300-900, 600-900 of LCG are all negative and significant. After 1024 (the period) integers, the LCG sequences start to repeat themselves which is why they have worse scores. This suggests us to use the LCG instead of professional models when the required sequence length is less than or equal to the LCG's period. On the other hand, the three professional models have extremely long periods so they are less likely to have the repetition issues. From [2.2.1](#) we can see that their scores increase steadily as length grows so that they could handle long sequences efficiently.

Contrast	300	600	900	1200	1500
LCG – MT	0.015005 (< .0001)	0.042827 (< .0001)	0.078134 (< .0001)	–0.134073 (< .0001)	–0.152612 (< .0001)
LCG – RANDU	0.015079 (< .0001)	0.042824 (< .0001)	0.078148 (< .0001)	–0.133883 (< .0001)	–0.152356 (< .0001)
LCG – XOR	0.014917 (< .0001)	0.042748 (< .0001)	0.078180 (< .0001)	–0.134045 (< .0001)	–0.152599 (< .0001)
MT – RANDU	0.000075 (0.9566)	–0.000003 (1.0000)	0.000015 (0.9994)	0.000190 (0.2605)	0.000256 (0.0256)
MT – XOR	–0.000087 (0.9327)	–0.000079 (0.9162)	0.000046 (0.9843)	0.000028 (0.9939)	0.000013 (0.9989)
RANDU – XOR	–0.000162 (0.6720)	–0.000076 (0.9307)	0.000032 (0.9945)	–0.000162 (0.4129)	–0.000243 (0.0367)

Table 2.4.2: Pairwise Algorithm Comparisons at each Length (Tukey)

2.4.3 Contrasts

From 2.1.1d, we can observe a clear linear trend in the score of LCG as length grows from 300 to 900; we suspect this to be the major reason for the partial increasing trend in 2.1.1b. We want to test a linear contrast using coefficients $(-1, 0, 1)$. Define the contrast by

$$\hat{C}_1 = -\bar{y}_{LCG,300} + 0\bar{y}_{LCG,600} + \bar{y}_{LCG,900} \quad (2.4)$$

From 2.4.3, the linear contrast has a very small p-value and is thus significant. This indicates a strong positive linear trend in LCG’s score, which aligns with what we expected before. The confidence interval is fairly small so the estimate is precise. This might suggest that the LCG’s performance improves linearly over length, as long as the length does not exceed the period. In the future, we could examine more LCGs to verify this.

We now want to see whether the LCG family (LCG, RANDU) have the same performance as the digit shift family (MT, XOR)^{2.1}. The contrast is defined by

$$\hat{C}_2 = \frac{1}{10} \sum_L \bar{y}_{LCG,L} + \frac{1}{10} \sum_L \bar{y}_{RDU,L} - \frac{1}{10} \sum_L \bar{y}_{MT,L} - \frac{1}{10} \sum_L \bar{y}_{XOR,L} \quad (2.5)$$

From 2.4.3, the difference is significant due to the small p-value. But the difference -1.5% is not very huge - this might be due to the large variability over the LCG groups. Overall, the two families are competitive.

contrast	est.	s.e.	df	lower	upper	t	p-value
$\hat{C}_1$	0.1549	7.59e-05	19961	0.1547	0.155	2040.199	< .0001
$\hat{C}_2$	-0.01513	3.38e-05	19961	-0.01521	-0.01506	-447.593	< .0001

Table 2.4.3: Contrasts

^{2.1}Both of them involve binary digit shifts.

2.4.4 Residual Analysis

We check the adequacy of model (2.1) using the standard residual diagnostic plots in Figure 2.4.2. Due to the unequal variance across groups, the residuals are studentized according to their own group's standard deviation.

Normality In the QQ plot 2.4.2b, the left side departs notably from the reference line, showing a heavy left tail. The right side looks better but also contains many outliers away from the line. This shows strong evidence on non-normality. Similarly, the histogram 2.4.2a has minor^{2.2} left skewness and most data fall in the middle range $[-1, 1]$, which is not a smooth "bell shape" like normal or t distribution. Now look at the residual plot by groups D.1.6. Similar to the raw data plot D.1.2a, the length-300 groups and some LCG groups have large left tails. Apart from that, the residuals from other groups are not that severely skewed but only a few of them pass the Shapiro-Wilk test, as in the table of normality tests D.4.2. The residuals do not satisfy the normality assumption. However, the Shapiro-Wilk test is sensitive to such a large group sample size, so it would be considered as a reference rather than hard standard for normality.

Residuals versus Fitted Values The vertical bands in 2.4.2c seem to be floating around 0 but most of them lean towards the downside of 0. Clearly there are more than 5% dots outside the 95% confidence interval (approximately $[-2, 2]$) which means the residuals are not normally distributed. Most outliers are distributed on the lower side which should be the reason for the large left tail in the residual histogram. Other than that, this graph reveals no obvious trends. Similarly, the residual plot 2.4.2d also shows significant trends and no fanning shape, but those horrible gaps seem abnormal which exhibits potential non-randomness in the residuals; they will be addressed later. The LR test D.4.2 has small p-value ($< .0001$) which strongly favours this model over the model assuming constant variance. The residual variance is reasonably flat.

Residual Independence Refer to the residual ACF plot D.1.7. Except for some potential false positives, the residual ACF has almost no significant lags. The residuals are fairly stationary and have no obvious trends shown in 2.4.2d; the p-value of 0.2066 ($> .05$) from the Ljung box test at lag 10 (see D.4.1) also proves that there are no significant correlations between residuals. The runs test has p-value 0.5198 ($> .05$) (see D.4.1), indicating that the residuals are sufficiently random. The independence assumption on residuals is well satisfied.

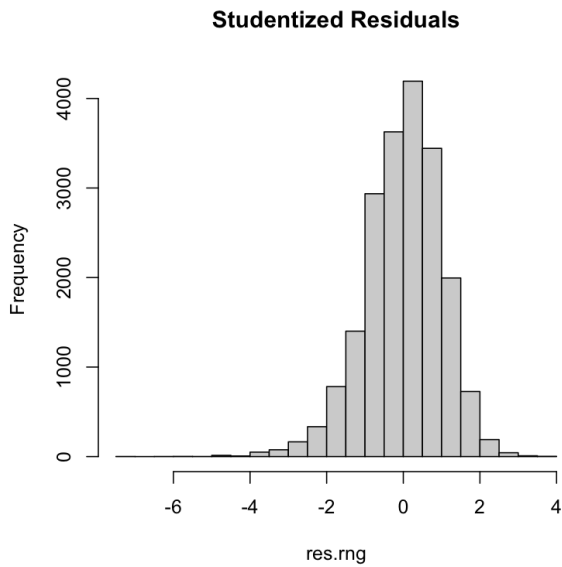
Overall Model Assessment and Analysis In general, the residuals are fairly random and independent but are not fully qualified for normality due to the severe left skewness in some groups. Thus the inferential results should not be interpreted as under the perfect normality assumption. Indeed, this is not surprising since we have observed the non-normality pattern and large tails in the raw data. We applied data transformations (in C.4) such as log, boxcox, but they have limited improvements on fulfilling normality. Nevertheless, this model still improves fit in comparison to the original model assuming constant variance C.1.

In fact, we did not expect the data to be normal. The response data are preference scores measured on the mechanical algorithms, not from natural sources such as the radioactive decay, atmospheric noise, or daily precipitations^{2.3}. That said, sequences from the same group might share some common properties and the scores won't necessarily behave randomly. For example, note that some LCG groups (like $(LCG, 900)$, $(LCG, 1200)$, $(LCG, 1500)$ in D.1.2a) have gaps in their residual histogram, which might be the reason for those notably many gap patterns in the studentized residual plot 2.4.2d; this is because the sequences generated from LCG at the same length have exactly the same entropy since they have the same integer frequency table^{2.4}. Consequently, we cannot expect smooth "random variability" in the scores of LCG sequences; they are instead fixed in some ranges. Additionally, as we can see in the raw residuals plot D.1.5, most residuals are within $[-0.5\%, 0.5\%]$, of which a large fraction lie in $[-0.25\%, 0.25\%]$ (the two large bands in the middle). Along with the almost perfect adjusted $R^2 = 0.998$, we would say such model fit is acceptable.

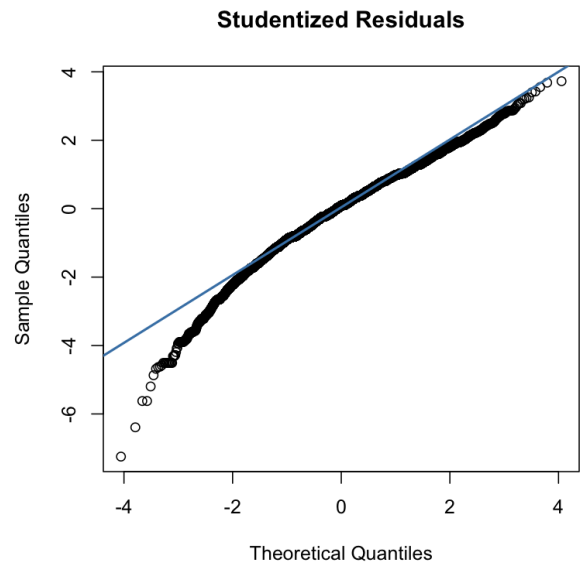
^{2.2}With such a large sample size, a small fraction could be a huge amount of data that lead to the significant imbalance in QQ plot.

^{2.3}These are called "truly-random" events, contrary to "pseudo-random" events like what we did.

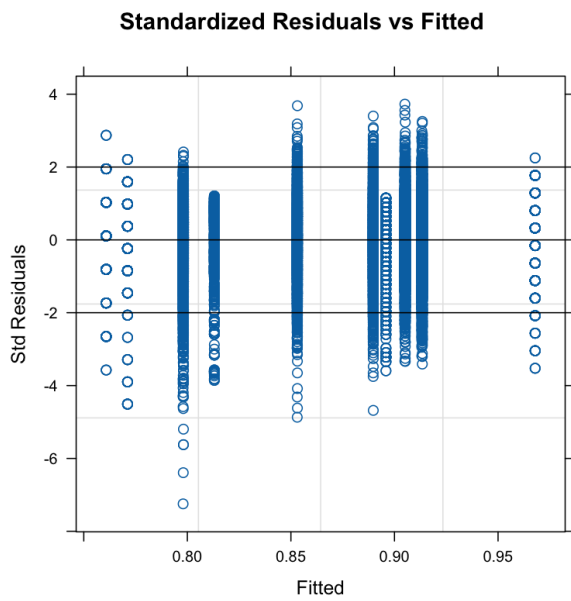
^{2.4}Consult A.1. The entropy evaluator only looks at the observed probabilities of all integers. For example, if we extract any two sequences from $(LCG, 1200)$, evaluate the frequency table and reorder everything, they will be the same table; their entropy should also be the same.



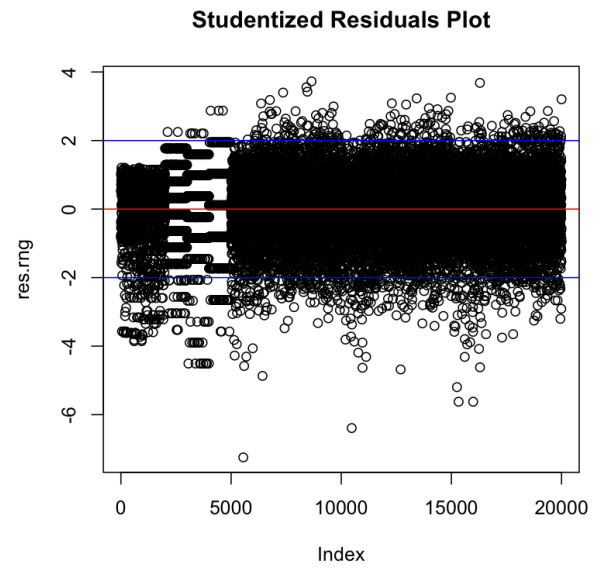
(a) Residual Histogram



(b) Residual QQ Plot



(c) Studentized Residuals versus Fitted Values



(d) Residual Plot

Figure 2.4.2: Residual Diagnostic Plots

Chapter 3

Conclusion

This study compared the performance of selected pseudo-random number generators (PRNG) for producing integer sequences over a restricted range $[1, 1024]$, using a comprehensive collection of randomness tests. By combining various measurements of randomness into a single evaluation score, we investigated how the choice of algorithm and the sequence length can affect the overall quality of the generated pseudo-random sequences.

The result shows that both algorithm and sequence length have a strong impact on the randomness of sequences. The full-period linear congruence generator (LCG) achieves the highest randomness scores for sequences whose lengths are close to while not exceeding its period. Although modern and professional models, such as Mersenne Twister, Xor Shift, RANDU, have superior performance for generating extremely long sequences of a broad number range, they did not show clear advantages for generating relatively short integer sequences within a limited range, in comparison to the small LCG model. However, once the sequence length exceeds the LCG's period, its performance drops rapidly due to the unavoidable repetition of numbers in the sequence. From this perspective, the professional models' performance improves over sequence length and are less likely to have repetition issues as they have sufficiently long periods. Additionally, the professional models themselves do not reveal significant differences under such experimental circumstances.

The conclusions suggest that the choice of PRNG should depend on the needs of the application rather than always using a single generator. Many applications need short and finitely ranged pseudo-random sequences, such as randomized controlled trials, A/B testing, or statistical simulations. Instead of blindly choosing the professional models, sometimes a small-period LCG with customized parameters^{3.1} might have provided sufficient randomness while needing much less computational time and memory^{3.2}. For longer sequences required by the more rigorous simulations, the professional generators are still the preferred choice for their excellent capabilities of producing stable and robust sequences.

The experiment outcome happens under a specific experimental setting, namely the range restricted to $[1, 1024]$ and lengths up to 1500, and does not apply to all conditions in general. The randomness score designed by us should be interpreted as a preference measurement rather than a rigorously proven mathematical evaluation for industrial-level comparisons and analysis. Nevertheless, it is a certainly effective and stable indicator of randomness in this experiment.

^{3.1}Make sure to satisfy the [Hull-Dobell Theorem](#) [6] and adjust its period to be close to the required length.

^{3.2}This is commonly agreed. In the future, we can include efficiency and costs as part of the overall score.

Appendix A

Randomness Tests

A.1 Entropy

Entropy [4] measures the uncertainty level of a sequence of integers. A large entropy means the data are fairly "messy", namely very random. Given a sequence of n integers between $[1, 1024]$, the observed probability p_i for each integer $i \in [1, 1024]$ is defined by

$$p_i = \frac{\text{count of integer } i}{n} \quad (\text{A.1})$$

The entropy is defined by

$$H = - \sum_{i=1}^{1024} p_i \log_2(p_i) \quad (\text{A.2})$$

where $0 \log(0) = 0$. We can normalized it by adjusting for the bin size (which here is 1024)

$$S_H = \frac{H}{\log_2(1024)} \quad (\text{A.3})$$

which gives a continuous index between $[0, 1]$.

A.2 The Runs Test (for reference only)

The centered test statistics Z should follow a $N(0, 1)$ distribution, thus we can score the difference between Z and 0 using the Gaussian function

$$S_R = \exp(-Z^2/2) \quad (\text{A.4})$$

Practically, this test causes very large variability in the overall score. Indeed, the discrepancy between the test statistics Z and 0 is subjective since Z is a single point estimate, and we need to consider whether it is in the confidence interval. Nevertheless, using the p-value as a pass/fail standard creates jumps in the final score and the weight is hard to be determined, which is also not a wise idea. Consequently, we decided to exclude runs test from the final score to maintain smoothness.

A.3 The Distribution Uniformity Test

We measure the total variation distance between the generated sequence and the standard uniform distribution. Specifically, it compares the observed probability p_i and the theoretical uniform probability $u_i = \frac{1}{1024}$ for each integer

$i \in [1, 1024]$. The total variation distance is

$$D_U = \frac{1}{2} \sum_{i=1}^{1024} \left| p_i - \frac{1}{1024} \right| \quad (\text{A.5})$$

The $\frac{1}{2}$ is to normalize the summation which is at most 2 since

$$\begin{aligned} \sum_{i=1}^{1024} \left| p_i - \frac{1}{1024} \right| &\leq \sum_{i=1}^{1024} |p_i| + \left| \frac{1}{1024} \right| \text{ by triangle inequality} \\ &= \sum_{i=1}^{1024} p_i + \sum_{i=1}^{1024} \frac{1}{1024} \\ &= 1 + 1 \text{ by property of probabilities} \\ &= 2 \end{aligned}$$

Hence, the normalized D is a continuous index between $[0, 1]$. A large such D reveals great discrepancy to the standard uniform distribution which implies poor randomness. The corresponding uniformity score can thus be defined by

$$S_U = 1 - D_U \quad (\text{A.6})$$

which is between $[0, 1]$ and higher the better.

A.4 Autocorrelation

Given a sequence, the autocorrelation ρ_L at lag L is measured as the average correlation between any two integers whose spacing is L . The plot of ρ_L at different lags look like [A.4.1](#). Since the numbers in 1 : 100 are highly correlated, the ACF reveals very significant lags and a smooth decay in the spikes, implying a potential trend. The ACF plot can also visually reflect repeated patterns if any, but unfortunately there is no numerical value to address that (except for differencing which is very similar to what we are doing for repetition score).

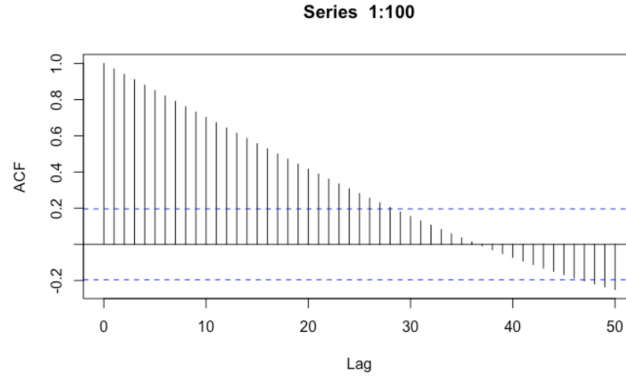


Figure A.4.1: Sample ACF plot for Sequence 1 : 100

The lags within the blue dashed lines are in the confidence interval and should be considered 0. However, since we are averaging the correlations, they cannot be simply dropped. The final score is chosen to be 1 minus the average correlation over 200 lags, not including lag 0.

$$S_{ACF} = 1 - \frac{1}{200} \sum_{L=1}^{200} |\rho_L| \quad (\text{A.7})$$

A.5 π -Test

We split each sequence in half to get two random sequences. Following tactics used by Monte Carlo estimator (E.2), calculate the estimated $\hat{\pi}$ and compare with the real π . We score the difference using Gaussian function

$$S_{\pi} = \exp(-(\hat{\pi} - \pi)^2/2) \quad (\text{A.8})$$

This test is naive - a high score does not mean the sequences are uniformly distributed. However, it can test how "uniform" the sequence is if told that the sequence is uniformly behaved.

A.6 Repetition Test

Except for ACF test, the above tests can only provide partial information but cannot perfectly detect repeated patterns such as

$$1, 2, 3, 1, 2, 3, 1, 2, 3, \dots$$

However, if we observe such repeated patterns, the sequence must be following some underlying rule and is thus not random. The largest index before the sequence repeats itself is called the period. We introduce a way to find the period. Suppose we shift the sequence by 3 digits to the right and count the match rate of digits after the shift

$$\begin{array}{c} 1, 2, 3, 1, 2, 3, 1, 2, 3 \text{ (original)} \\ \underbrace{1, 2, 3, 1, 2, 3}_{\text{match rate}=1} \text{ (shifted)} \end{array}$$

Since we found the match rate to be 1, the period is 3. In practice, we look at many possible options for lags until getting the period if any. Let p_i be the observed probability of integer i in the sequence, and the probability that two integers match is p_i^2 . Define the fraction to be

$$R = \frac{\text{match rate} - \text{chance}}{1 - \text{chance}} \quad (\text{A.9})$$

Define F to be the maximum rate R over all examined lags. F is 1 if there is a perfect match and 0 if there is no match at all. Something in between means a partial match is detected. The score is thus defined by

$$S_F = 1 - F \quad (\text{A.10})$$

We had considered two related testing standards:

- (Smooth) Accept repetitions and penalize short periods;
- (strict) Reject repetitions and assign score 0 if observed repeated patterns.

We finally chose the strict one since the repeated pattern is definitely not random and should not be tolerated.

A.7 Spectral Periodicity (Not Used)

Spectral periodicity refers to recurring, rhythmic patterns in frequency data upon *Fast Fourier Transformation* (FFT) on the original time domain data. This occurs when the frequency components exhibit repeated or harmonic structures, which suggest potential cycles in the time domain data. We measure the spectral flatness using

$$F = \frac{\text{geometrical mean of power spectrum}}{\text{arithmetic mean of power spectrum}} \quad (\text{A.11})$$

It is close to 1 if being very flat like noise spectrum, and close to 0 if there exist strong periodic components. However, since our sequences are not sinusoidal, the FFT cannot accurately allocate frequency bins; this test is thus not suitable (and not fair) for our experiment.

A.8 Overall Score

Given the weight factor for each individual test such that

$$W_H + W_R + W_U + W_{ACF} + W_\pi + W_F = 1, \quad 0 \leq W \leq 1 \quad (\text{A.12})$$

we can calculate the overall randomness score by

$$S = W_H S_H + W_R S_R + W_U S_U + W_{ACF} S_{ACF} + W_\pi S_\pi + W_F S_F \quad (\text{A.13})$$

which gives a final score in $[0, 1]$.

As stated in the main content, the final weights are

$$W_H = W_U = W_{ACF} = W_\pi = W_F = 1/5, \quad W_R = 0$$

Appendix B

PRNG Algorithms

B.1 Linear Congruence Generator (LCG)

A LCG relation is defined by

$$x_{(n+1)} = ax_{(n)} + c \pmod{m}$$

Our LCG model is

$$x_{(n+1)} = 205x_{(n)} + 1 \pmod{2^{10}} \quad (\text{LCG})$$

For example, a sequence of length 10 given seed $x_0 = 1$ is

$$[1, 2, 207, 248, 461, 94, 635, 948, 601, 122] \mapsto 56\%$$

Our score function gives an evaluation of 56%. To give you a sense why the length can affect the randomness score, consider a longer sequence of length 20 with the same seed $x_0 = 1$,

$$[1, 2, 207, 248, 461, 94, 635, 948, 601, 122, 231, 48, 421, 86, 19, 620, 945, 1010, 1023, 616] \mapsto 63\%$$

Note it also shows that the algorithm generates a unique sequence based on the same seed. The score increased to 63 which does reveal the potential effect of length. More experiments are listed below:

- LCG(seed 1, length 30): 67%
- LCG(seed 1, length 50): 70%
- LCG(seed 1, length 100): 74%

Therefore, we do have the reason to investigate the effect of length.

The period of a sequence is the smallest number of integers before they start to repeat. For example, the sequence $[1, 2, 3, 1, 2, 3, 1, 2, \dots]$ has period 3 since it repeats itself after 3 integers. A LCG has a full period, which is at most the modulus m , when the following theorem is satisfied.

Theorem B.1 (Hull-Dobell Theorem [6]). *A LCG*

$$x_{(n+1)} = ax + c \pmod{m}$$

has full period m if and only if

- *c is coprime to m , i.e. $\gcd(c, m) = 1$.*
- *$a - 1$ is divisible by all prime factors of m .*

- $4 \mid a - 1$ if $4 \mid m$.

The parameters of (LCG) are intentionally chosen to fulfill theorem B.1 so LCG has full period 1024. The sequence starts to repeat after 1024 numbers which is why LCG performs poorly at length 1200, 1500.

The RANDU [1] is a historically important but structurally flawed LCG. The implementation can be found here [7]. The algorithm is

$$x_{(n+1)} = (2^{16} + 3)x_{(n)} \pmod{2^{31}} \quad (\text{RANDU})$$

based on the startup seed x_0 , and the n -th integer in the sequence is mapped to $y_{(n)} = (x_{(n)} \ggg 21) + 1$ ^{B.1}. However, RANDU does not satisfy (B.1), which does not have full period 2^{31} . In the 3-D perspective, RANDU's generated numbers evenly distribute on 15 parallel planes (as in Figure (B.1.1)), which are not necessarily random but are enough for most industrial applications.

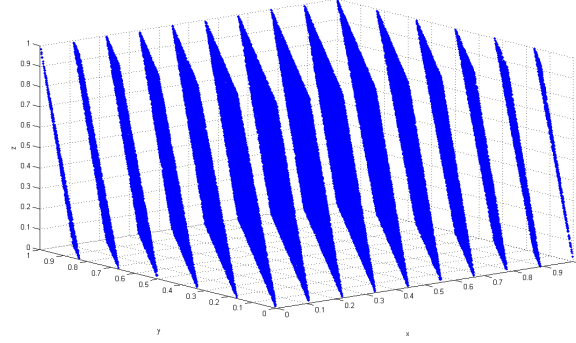


Figure B.1.1: RANDU Distribtuions

Other typical LCGs include the composition of multiple LCGs, combination with other algorithms like Xor Shift, or integration with cryptography.

B.2 Mersenne Twister

We used an implementation of MT19937, which has a period of $2^{19937} - 1$. More details can be found here [2]. All generated integers are mapped to $[1, 1024]$. The implementation can be found here [7].

B.3 Xor Shift

Xor shift applies digit shifts and xor operations to generate sequences. More details are here [3] and the implementation is here [7].

^{B.1}This maps the integer to be within $[1, 1024]$. Note that $x \ggg 6$ means convert x to binary form and shift it by 6 digits to the right.

Appendix C

Other Fitted Models

C.1 Model Assuming Constant Variance

The original model does not account for unequal variance, which has much worse residuals. It was fitted using `lm`, which gives wrong variance and confidence intervals.

$$y_{ijk} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \epsilon_{ijk} \sim N(\mu_{ij}, \sigma^2) \quad (\text{C.1})$$

for algorithm $i = 1, 2, 3, 4$, length $j = 1, 2, 3, 4, 5$, and seed choice $k = 1, 2, 3, 4$, with constraints

$$\sum_{i=1}^4 \alpha_i = \sum_{j=1}^5 \beta_j = \sum_{i=1}^4 (\alpha\beta)_{ij} = \sum_{j=1}^5 (\alpha\beta)_{ij} = 0$$

The parameters are:

- y_{ijkl} : the randomness score of the l -th sequence generated by algorithm i at length j and with seed level k
- μ : grand mean of all data
- α_i : effect of algorithm $i \in \{1, 2, 3, 4\}$
- β_j : effect of length $j \in \{1, 2, 3, 4, 5\}$
- $(\alpha\beta)_{ij}$: effect of algorithm i , length j interaction
- ϵ_{ijk} : iid $N(0, \sigma^2)$ noise

R code

```
lm.rng <- lm(score ~ algo * len, data=data.rng)
```

model	residual s.e.	residual df	adj R^2	AIC	BIC	F p-value
const var	0.002391	19980	0.9981	-184666.7	-184500.7	< 2.2e-16

Table C.1.1: Model assuming Const Var

The residuals [C.1.1](#) are terrible. No need to look at this model.

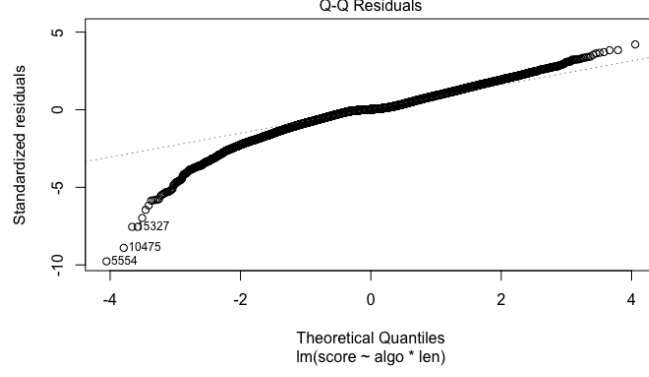


Figure C.1.1: Residual QQ plot of Model assuming Const Var

C.2 Model with Blocking Factor Seed

$$y_{ijkl} = \mu + \alpha_i + \beta_j + (\alpha\beta)_{ij} + \delta_k + \epsilon_{ijkl} \sim N(\mu_{ijk}, \sigma_{ijk}^2) \quad (\text{C.2})$$

for algorithm $i = 1, 2, 3, 4$, length $j = 1, 2, 3, 4, 5$, and seed choice $k = 1, 2, 3, 4$, with constraints

$$\sum_{i=1}^4 \alpha_i = \sum_{j=1}^5 \beta_j = \sum_{i=1}^4 (\alpha\beta)_{ij} = \sum_{j=1}^5 (\alpha\beta)_{ij} = \sum_{k=1}^4 \delta_k = 0$$

The parameters are:

- y_{ijkl} : the randomness score of the l -th sequence generated by algorithm i at length j and with seed level k
- μ : grand mean of all data
- α_i : effect of algorithm $i \in \{1, 2, 3, 4\}$
- β_j : effect of length $j \in \{1, 2, 3, 4, 5\}$
- $(\alpha\beta)_{ij}$: effect of algorithm i , length j interaction
- δ_k : (blocking) effect of seed $k \in \{1, 2, 3, 4\}$
- ϵ_{ijk} : iid $N(0, \sigma_{ijk}^2)$ noise

In general this model has no big differences compared with the model we used. We thus abandoned it to avoid overfitting.

C.3 Linear Model

We also tried to fit a linear model for length.

$$y_{ijk} = \mu + \alpha_i + (\beta + \gamma_i)L + \delta_k + \epsilon_{ijk}$$

with $\sum_{i=1}^4 \alpha_i = \sum_{i=1}^3 \gamma_i = 0$. Parameters:

- μ : grand mean and intercept
- α_i : effect of algorithm i for $i = 1, 2, 3$

- L : length of sequence
- β : common slope for length over all algorithms
- γ_i : adjustment for slope at algorithm i (algorithm-length interaction effect)
- δ_k : (blocking) effect of seed k for $k = 1, 2, \dots, 1000$
- ϵ_{ijk} : iid $N(0, \sigma^2)$ noise

This model has very poor fit and it is not worth to show the results. We also tried higher-order models which has limited improvements as well. In general, the linear model is too sensitive and is not suitable for explaining randomness score which are preference scores, not some deterministic quantitative measurements.

C.4 Data Transformations

The data are observed to be non-normal. To fulfill the normality, we first applied log transformation on data.

```
# R code
gls.trans <- gls(log(score) ~ algo * len,
  weights = varIdent(form = ~ 1 | algo * len),
  data = data.rng)
```

Residuals [C.4.1](#) seems no different.

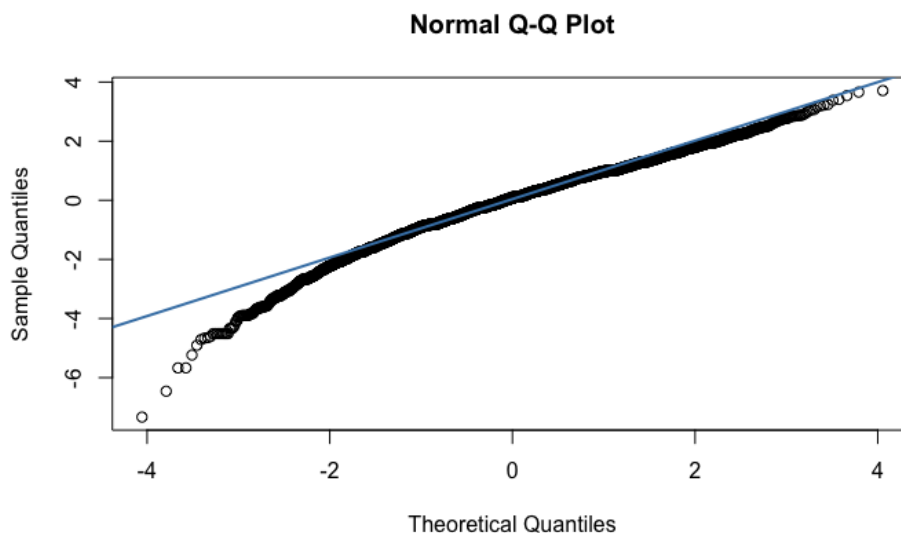


Figure C.4.1: Log Transformation

We also tried $y^2, y^3, \ln(y)^{1/3}$, but they turn out to be the same plot.

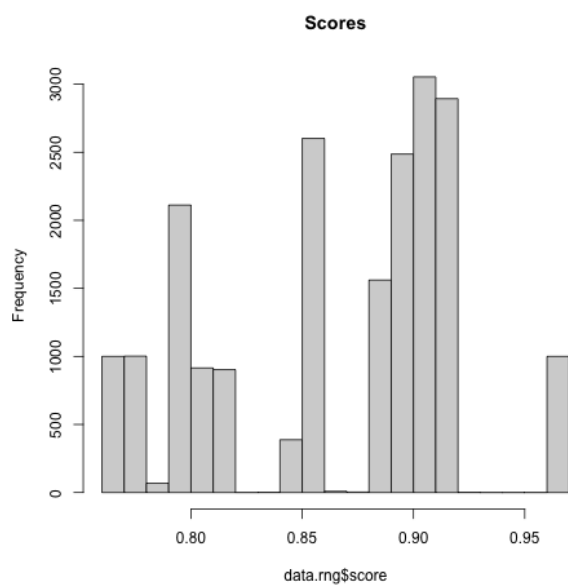
Appendix D

R Code and Output

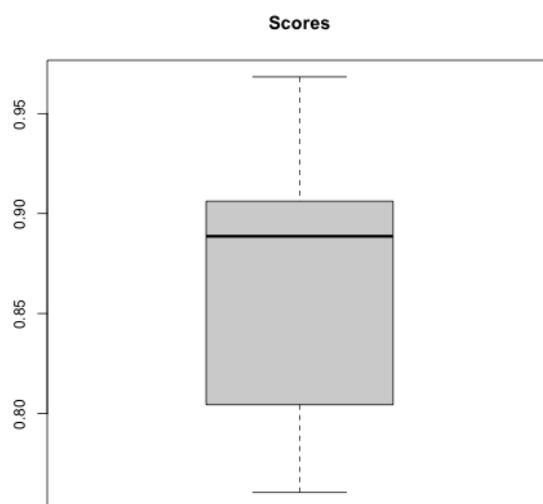
D.1 Graphics

D.1.1 Raw Data

Figure D.1.1: Raw Data Plot

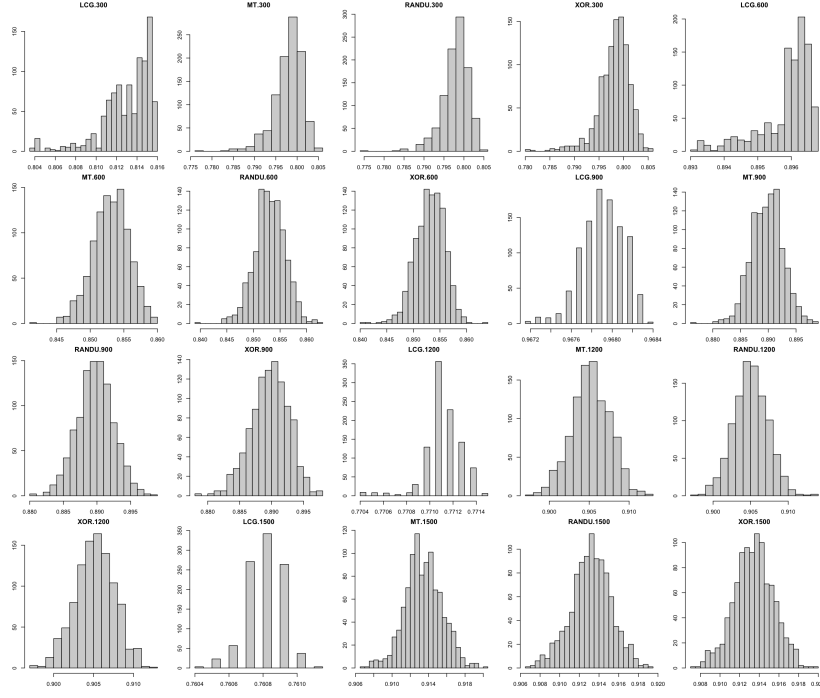


(a) Histogram of Raw Score Data

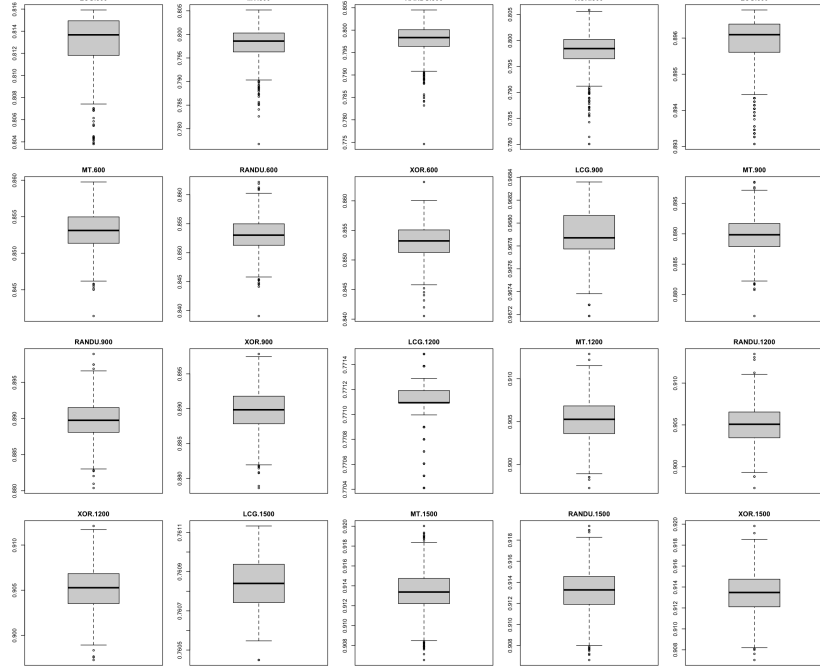


(b) Box Plot of Raw Score Data

Figure D.1.2: Raw Data Plots by Treatment Groups



(a) Histogram of Data by Treatment Groups



(b) Box Plots of Data by Treatment Groups

D.1.2 Seed

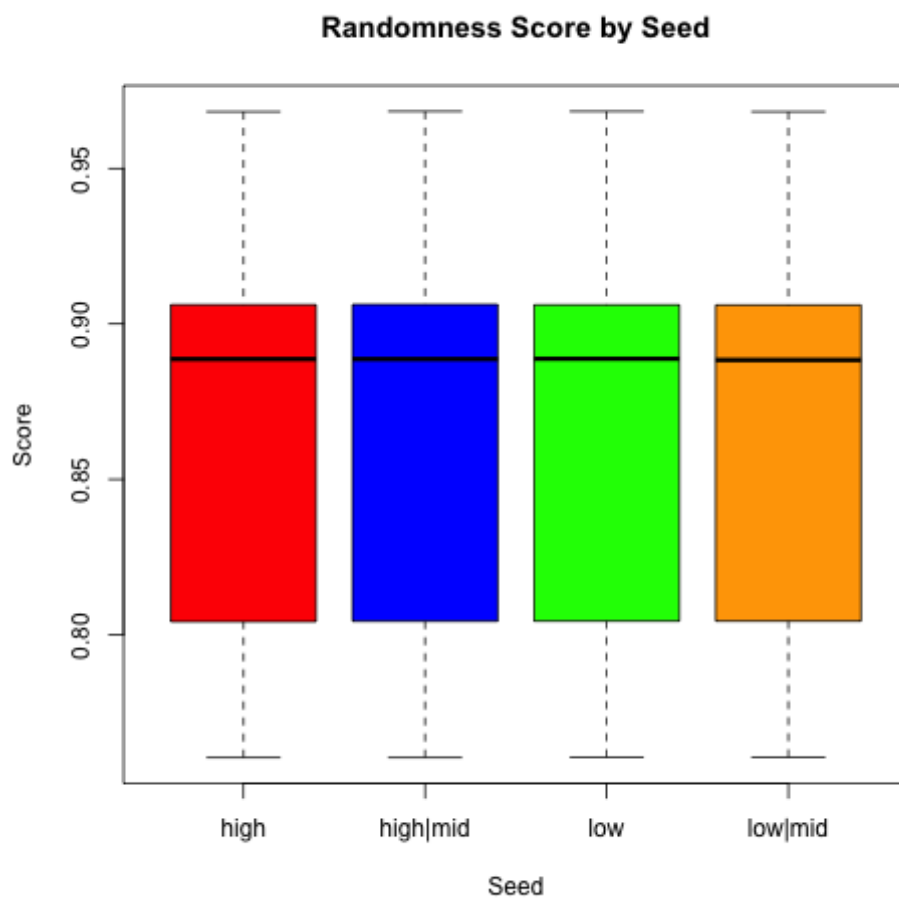
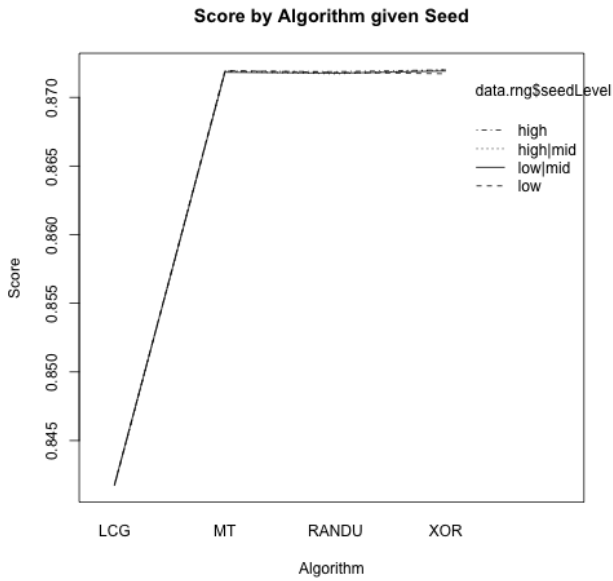
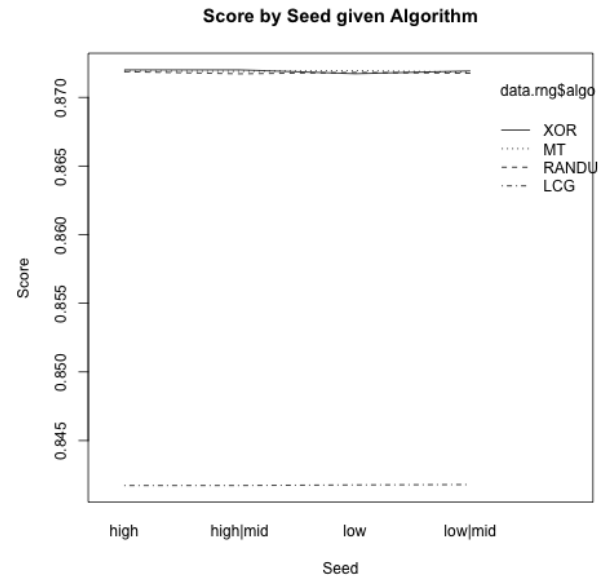


Figure D.1.3: Seed Box Plot

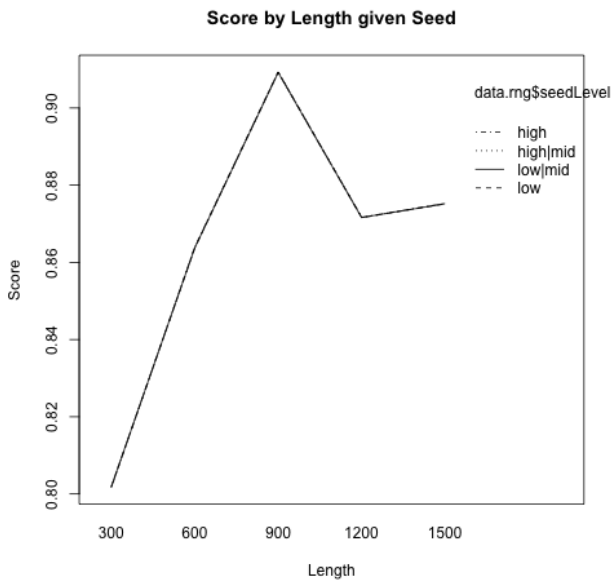
Figure D.1.4: Seed Interactions



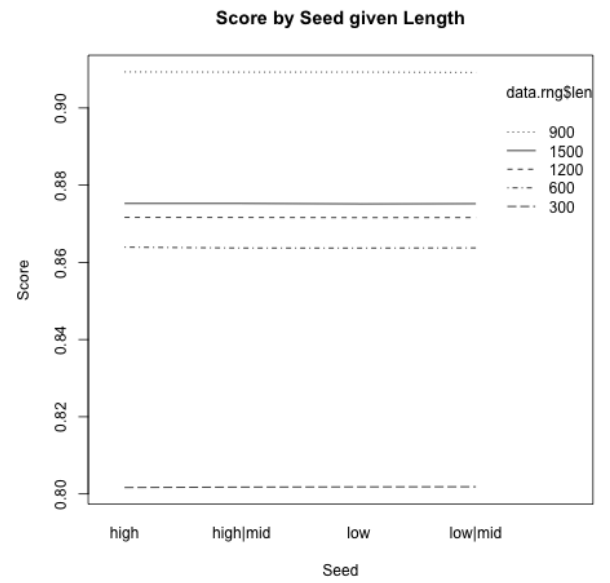
(a) Interaction of Algo by Seed



(b) Interaction of Seed by Algo



(c) Interaction of Length by Seed



(d) Interaction of Seed by Length

D.1.3 Residuals

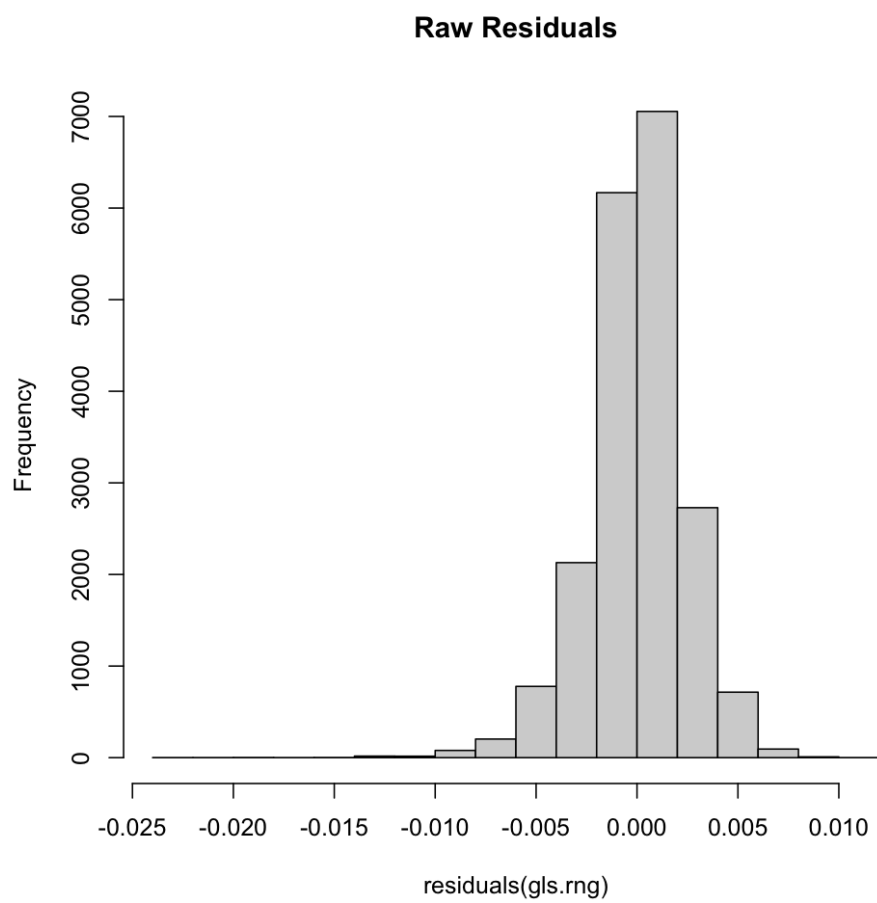


Figure D.1.5: Raw Residuals Histogram

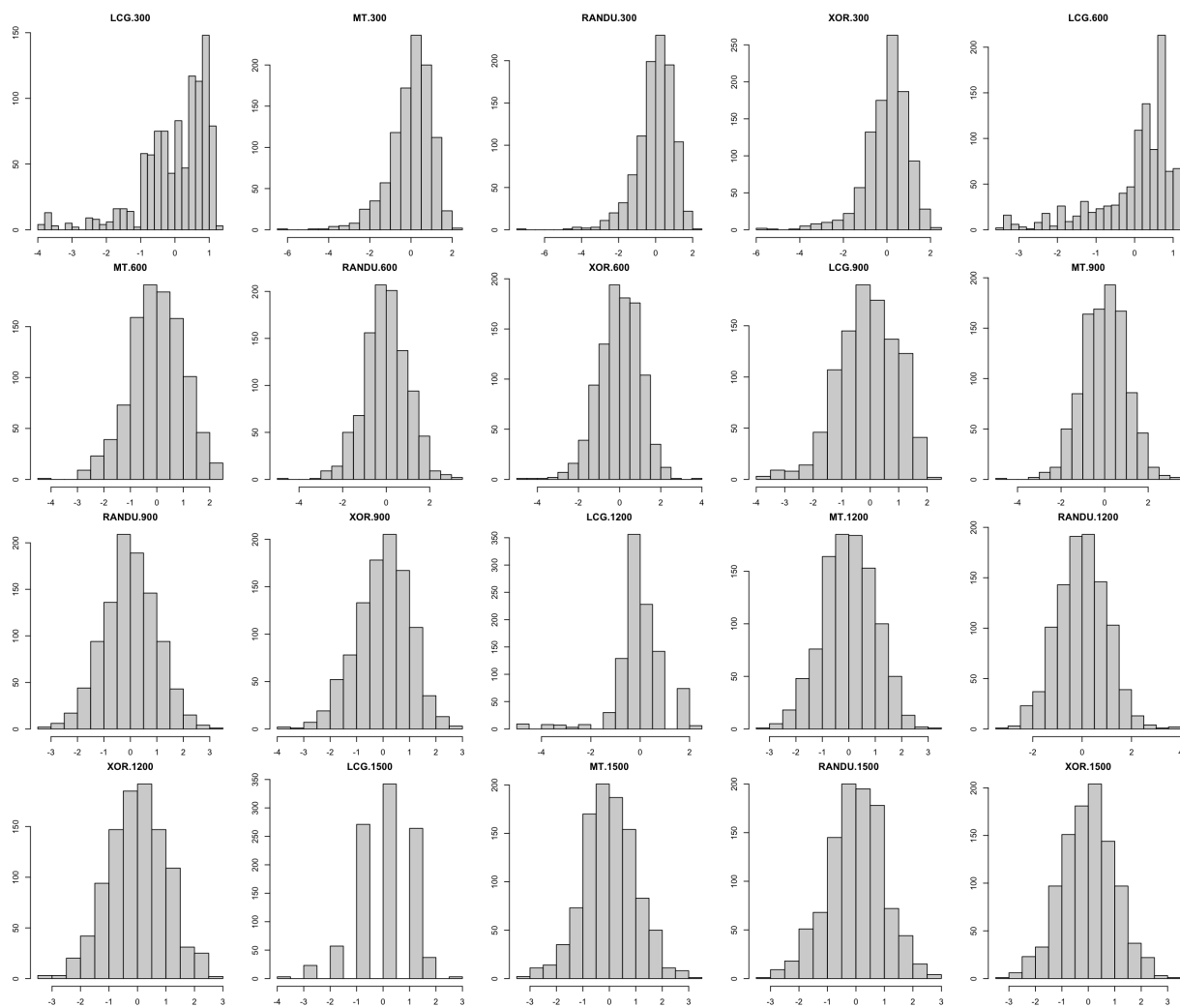


Figure D.1.6: Studentized Residual Histograms by Groups

Studentized Residuals

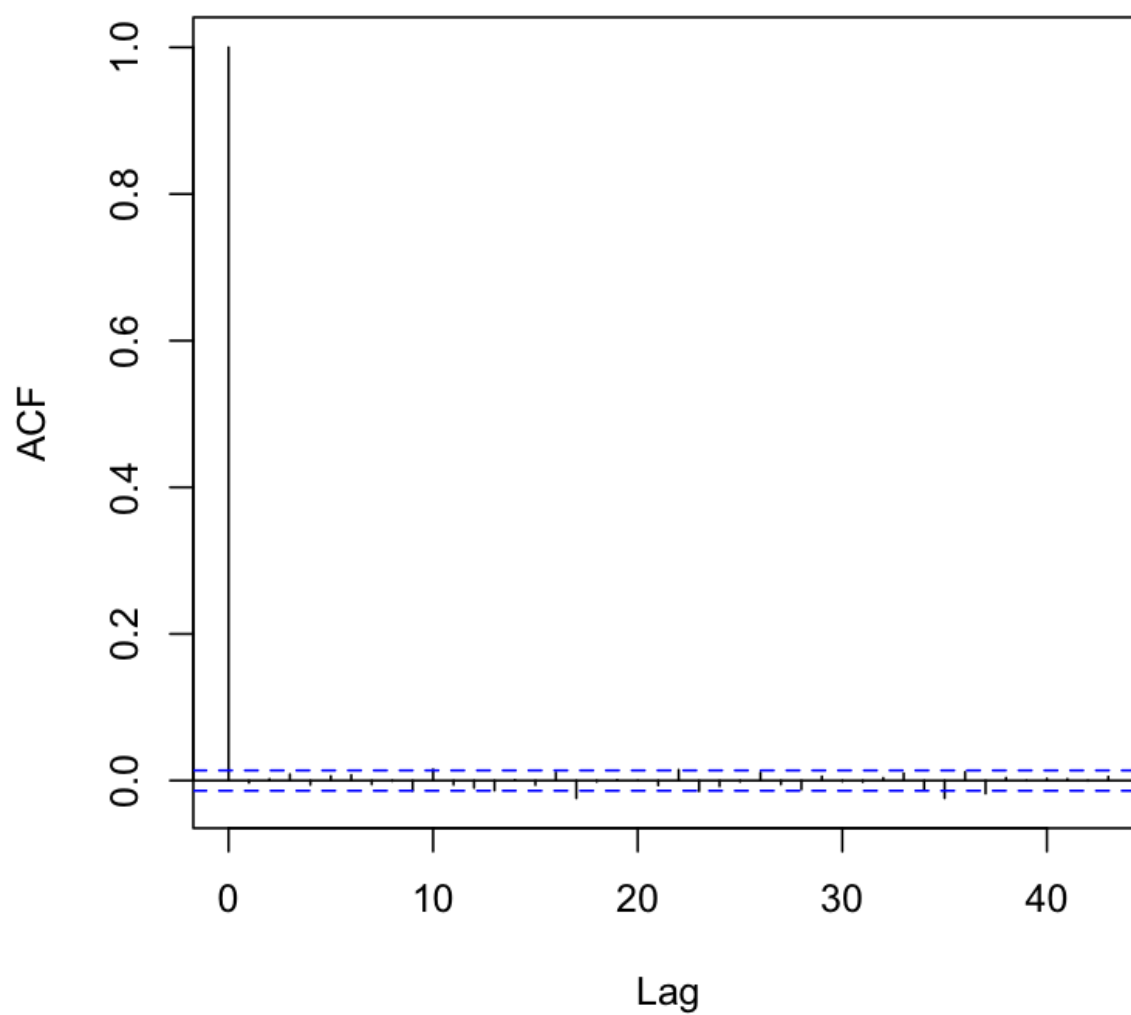


Figure D.1.7: Residual ACF

D.2 Model Fitting

D.2.1 Testing Equal Variances

```
# equal variance
bartlett.test(score ~ interaction(algo,len), data=data.rng)

library(car)
leveneTest(score ~ algo * len, data=data.rng)
```

Bartlett test of homogeneity of variances

data: score by interaction(algo, len)
Bartlett's K-squared = 16122, df = 19, p-value < 2.2e-16

Levene's Test for Homogeneity of Variance (center = median)
Df F value Pr(>F)
group 19 272.66 < 2.2e-16 ***
19980

Figure D.2.1: Tests for Equal Variance

Group	Relative_SD	Relative_Variance	Estimated_SD	Estimated_Variance
LCG*1200	0.0668889228449184	0.00447412799935345	0.000159988614169839	2.55963566639855E-08
LCG*1500	0.0443420328703852	0.00196621587907832	0.000106059719407566	1.12486640808116E-08
LCG*300	1	1	0.00239185514379969	5.72097102892103E-06
LCG*600	0.327162899180647	0.107035562600286	0.000782526263265649	6.123473527005E-07
LCG*900	0.0848339570045272	0.00719680026104597	0.00020291053643016	4.11726857943753E-08
MT*1200	0.994702246652846	0.98943255949622	0.00237918368520572	5.66051500794906E-06
MT*1500	0.845888969209147	0.715528148229714	0.00202324388208632	4.09351580639971E-06
MT*300	1.39147693870802	1.93620807095624	0.00332821127332742	1.10769902799037E-05
MT*600	1.12869078916593	1.27394289754801	0.00269966486982586	7.2881904093719E-06
MT*900	1.1882468207682	1.41193050706573	0.00284211427035804	8.07761352577282E-06
RANDU*1200	0.949397901945026	0.901356376217617	0.00227082225527984	5.15663371507423E-06
RANDU*1500	0.858562130123046	0.737128931281422	0.00205355624720643	4.21709326044054E-06
RANDU*300	1.3468064741797	1.81388767889235	0.00322136599296943	1.03771988606599E-05
RANDU*600	1.2016623482007	1.44399239908323	0.00287420226865426	8.26103868113731E-06
RANDU*900	1.12790521313478	1.2721701698166	0.0026977858857549	7.27804868537834E-06
XOR*1200	1.01632271098431	1.03291185286249	0.00243089670402826	5.90925878565544E-06
XOR*1500	0.834875730658013	0.69701748564175	0.00199690181080789	3.98761684200783E-06
XOR*300	1.34129597952974	1.79907490470264	0.00320818568799605	1.02924554086627E-05
XOR*600	1.14113218430559	1.30218266205806	0.00272942288478671	7.4497492839974E-06
XOR*900	1.23847515723262	1.53382071508236	0.00296225317529496	8.7749438745451E-06

Figure D.2.2: Group Variances

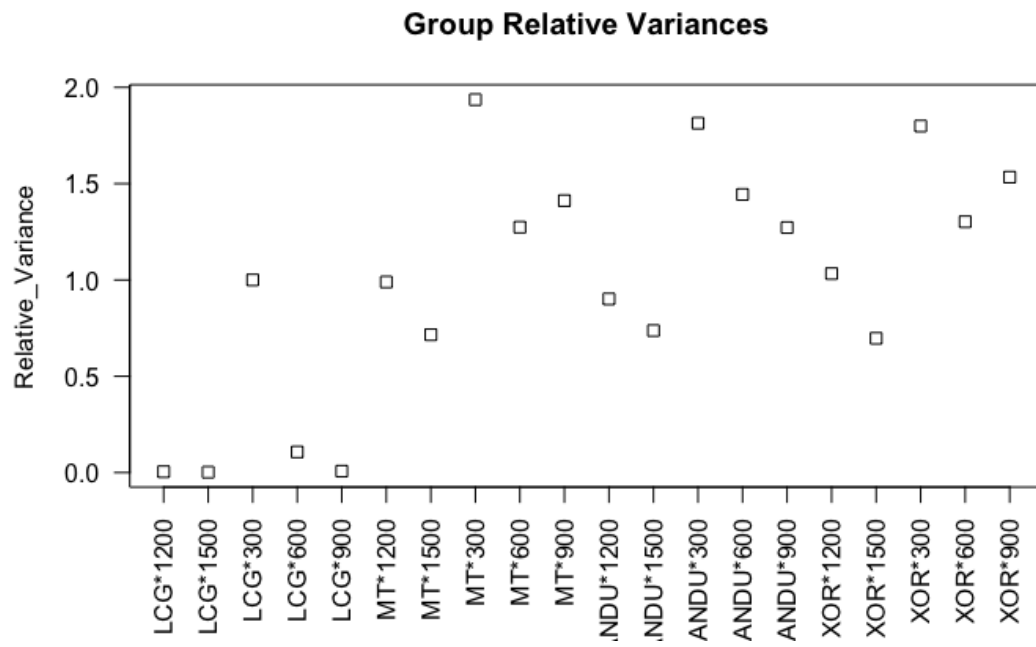


Figure D.2.3: Relative Group Variances

D.2.2 Model

Note that we used `gls` instead `lm` since the variances are different across groups.

```
library(nlme)
library(emmeans)

# The gls handles non-constant variance across groups
# model with seed
gls.rng.seed <- gls(score ~ algo * len + seedLevel,
  weights = varIdent(form = ~ 1 | algo * len),
  data = data.rng)
summary(gls.rng.seed) # output omitted

# model without seed
gls.rng <- gls(score ~ algo * len,
  weights = varIdent(form = ~ 1 | algo * len),
  data = data.rng)
gls.summary <- summary(gls.rng)
gls.summary
```

```
library(performance)

# get R2
r2(gls.rng)
r2(gls.rng.seed)

# Anova Test
model.seed.ML <- gls(score ~ algo * len + seedLevel,
  weights = varIdent(form = ~ 1 | algo * len),
  data = data.rng, method='ML')
model.ML <- gls(score ~ algo * len,
  weights = varIdent(form = ~ 1 | algo * len),
  data = data.rng, method='ML')
anova(model.ML, model.seed.ML)
```

Note they use the maximum likelihood method, which additionally counts the 20 variance estimators towards total model df.

Model	df	AIC	BIC	LogLik	L Ratio	p-value
No Seed	40	-200772.0	-200455.9	100426.0		
With Seed	43	-200770.6	-200430.7	100428.3	4.595392	0.2039

Table D.2.1: ANOVA Model Adequacy Test

Term	Value	Std.Error	t-value	p-value
(Intercept)	0.813033564799937	7.56371008759576E-05	10749.1370687685	0
algoMT	-0.0150045707999728	0.000129606949307602	-115.769801543294	0
algoRANDU	-0.0150793777999775	0.0001268785635542	-118.848900693425	0
algoXOR	-0.0149171639999704	0.000126544167931925	-117.881086451927	0
len600	0.0828456218000292	7.95821486366158E-05	1041.00760307836	0
len900	0.15486903680003	7.59087854909742E-05	2040.19911263689	0
len1200	-0.0419020941999714	7.58061170723346E-05	-552.75346922186	0
len1500	-0.05220539979997	7.57114237945699E-05	-689.531343930613	0
algoMT:len600	-0.0278220740000281	0.000157157561290874	-177.032996513186	0
algoRANDU:len600	-0.0277443386000154	0.000158023909341008	-175.570511549265	0
algoXOR:len600	-0.0278307750000296	0.00015516289206599	-179.364889565176	0
algoMT:len900	-0.0631291654000276	0.000157850396009606	-399.930358085295	0
algoRANDU:len900	-0.0630691010000218	0.000153027419963723	-412.142484104961	0
algoXOR:len900	-0.0632630644000287	0.00015757392867452	-401.481799255655	0
algoMT:len1200	0.14907713079997	0.000149946899512583	994.199488516003	0
algoRANDU:len1200	0.148961896199979	0.000145878031112706	1021.14002405812	0
algoXOR:len1200	0.148961792199971	0.000148149524399853	1005.48275671764	0
algoMT:len1500	0.167617025999974	0.000144577749945505	1159.35561359305	0
algoRANDU:len1500	0.167435784599982	0.000142571076358787	1174.40219205908	0
algoXOR:len1500	0.167516637799971	0.000141464808145602	1184.15767140867	0

Figure D.2.4: Model Estimates

D.3 Comparison Tests and Contrasts

With unequal variances, the contrasts cannot be directly calculated via TukeyHSD. In the vectorized form,

$$Y = X\beta + E \sim N(\mu, \Sigma) \quad (\text{D.1})$$

where $\Sigma = V(Y) = \sigma^2 \text{diag}(\delta_{1,1}^2, \dots, \delta_{1,1}^2, \delta_{1,2}^2, \dots, \delta_{1,2}^2, \dots, \delta_{3,5}^2)$, not $\sigma^2 I$ anymore. If $\hat{\mu}$ denotes the vector of groups means and c the contrast vector, then

$$\hat{t} = \frac{c^T \hat{\mu}}{\sqrt{c^T \hat{\Sigma} c}} \sim t(19980) \quad (\text{D.2})$$

`gls` handles this well. Basic setup:

```
rng.means <- emmeans(gls.rng, ~ algo * len)
group.means <- as.data.frame(rng.means)
group.means

# set a name vector to get the row index of each combination
combo.names <- paste(group.means$algo, group.means$len, sep = ".")
combo.names

lens.rng <- emmeans(gls.rng, ~ len | algo)
algos.rng <- emmeans(gls.rng, ~ algo | len)
lens.rng
algos.rng
```

```
# test best combo (LCG,900)
LCG.900.all.comps <- as.data.frame(contrast(rng.means, method = "trt.vs.ctrl",
  ref = match('LCG.900', combo.names),
  adjust = "dunnett"))

LCG.900.all.comps

write.csv(LCG.900.all.comps, file='LCG900_comparisons.csv', row.names = FALSE)
```

Outputs [D.3.1](#).

```
# compares lengths within each algo
comp.by.algo <- as.data.frame(pairs(rng.means, by='algo', adjust='tukey'), infer=TRUE)
comp.by.algo
write.csv(comp.by.algo, file='comp_by_algo.csv', row.names = FALSE)

# compares algos at each length
comp.by.len <- as.data.frame(pairs(rng.means, by='len', adjust='tukey'), infer=TRUE)
comp.by.len
write.csv(comp.by.len, file='comp_by_len.csv', row.names = FALSE)
```

Outputs [2.4.2](#) and [D.3.1](#).

contrast	estimate	SE	df	t.ratio	p.value
LCG len300 - LCG len900	-0.15486903680003	7.59087854909742E-05	19961	-2040.19911263689	0
MT len300 - LCG len900	-0.169873607600003	0.000105442699916578	19961	-1611.05138368421	0
RANDU len300 - LCG len900	-0.169948414600007	0.000102070424445342	19961	-1665.01134411382	0
XOR len300 - LCG len900	-0.1697862008	0.000101654454375876	19961	-1670.22883396925	0
LCG len600 - LCG len900	-0.0720234150000006	2.55640379927519E-05	19961	-2817.37239713152	0
MT len600 - LCG len900	-0.114850059800002	8.56116995227063E-05	19961	-1341.52295118894	0
RANDU len600 - LCG len900	-0.114847131399993	9.11164714359107E-05	19961	-1260.44314041259	0
XOR len600 - LCG len900	-0.114771354000001	8.6550112477061E-05	19961	-1326.06822469953	0
MT len900 - LCG len900	-0.0781337362000004	9.01043073974102E-05	19961	-867.147625422468	0
RANDU len900 - LCG len900	-0.0781484787999992	8.55524480723541E-05	19961	-913.456956064037	0
XOR len900 - LCG len900	-0.0781802283999991	9.38941774570685E-05	19961	-832.641922186769	0
LCG len1200 - LCG len900	-0.196771131000001	8.17123261560718E-06	19961	-24080.9606404014	0
MT len1200 - LCG len900	-0.0626985710000035	7.5509520550348E-05	19961	-830.339943136013	0
RANDU len1200 - LCG len900	-0.0628886125999992	7.2095814031529E-05	19961	-872.292149617684	0
XOR len1200 - LCG len900	-0.0627265028000009	7.71390398660096E-05	19961	-813.161570444183	0
LCG len1500 - LCG len900	-0.2070744366	7.24025896464937E-06	19961	-28600.4185224648	0
MT len1500 - LCG len900	-0.0544619813999987	6.43015434666548E-05	19961	-846.977824540734	0
RANDU len1500 - LCG len900	-0.0547180297999957	6.52553901699708E-05	19961	-838.52122648369	0
XOR len1500 - LCG len900	-0.0544749627999993	6.34727463388991E-05	19961	-858.241780009674	0

Figure D.3.1: LCG900 Comparisons

Contrast	LCG	MT	RANDU	XOR
300 – 600	−0.082846 (< .0001)	−0.055024 (< .0001)	−0.055101 (< .0001)	−0.055015 (< .0001)
300 – 900	−0.154869 (< .0001)	−0.091740 (< .0001)	−0.091800 (< .0001)	−0.091606 (< .0001)
300 – 1200	0.041902 (< .0001)	−0.107175 (< .0001)	−0.107060 (< .0001)	−0.107060 (< .0001)
300 – 1500	0.052205 (< .0001)	−0.115412 (< .0001)	−0.115230 (< .0001)	−0.115311 (< .0001)
600 – 900	−0.072023 (< .0001)	−0.036716 (< .0001)	−0.036699 (< .0001)	−0.036591 (< .0001)
600 – 1200	0.124748 (< .0001)	−0.052151 (< .0001)	−0.051959 (< .0001)	−0.052045 (< .0001)
600 – 1500	0.135051 (< .0001)	−0.060388 (< .0001)	−0.060129 (< .0001)	−0.060296 (< .0001)
900 – 1200	0.196771 (< .0001)	−0.015435 (< .0001)	−0.015260 (< .0001)	−0.015454 (< .0001)
900 – 1500	0.207074 (< .0001)	−0.023672 (< .0001)	−0.023430 (< .0001)	−0.023705 (< .0001)
1200 – 1500	0.010303 (< .0001)	−0.008237 (< .0001)	−0.008171 (< .0001)	−0.008252 (< .0001)

Table D.3.1: Pairwise Sequence Length Comparisons within Each Algorithm (Tukey)

```
# constructing contrast vector (aligning with the row)
# linear contrast
con.vec <- -1*(combo.names=='LCG.300')+(combo.names=='LCG.900')
con.vec

# family contrast
con.vec.2 <- 0.1*(group.means$algo=='LCG')+0.1*(group.means$algo=='RANDU')+
(-0.1)*(group.means$algo=='MT')+(-0.1)*(group.means$algo=='XOR')
con.vec.2
```

```
# test and output confidence intervals
lin.con <- contrast(rng.means,
                    method=list('linear.contrast'=con.vec,
                                'family.constrast'=con.vec.2),
                    adjust='bonferroni')
summary(lin.con, infer = c(TRUE, TRUE), level = 0.95)
```

Outputs [2.4.3](#).

D.4 Residual Analysis

```
# Ljung Box test at lag 10
Box.test(res.rng, lag=10, type='Ljung-Box')
plot.ljung('studentized residuals', res.rng, 10)

# runs test
randtests::runs.test(res.rng)
```

Test	p-value
Ljung Box Test	0.2066
Runs Test	0.5198

Table D.4.1: Randomness Tests

Ljung-Box:studentized residuals

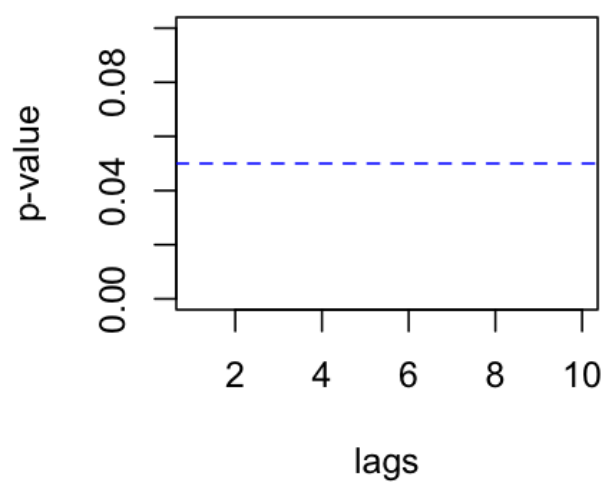


Figure D.4.1: Ljung Box Plot

Here we test whether the model accounting for unequal variance is adequate, comparing to the model assuming constant variance.

```
# model assuming constant variance
gls.rng.constvar <- gls(score ~ algo * len, data=data.rng)

# comparing with our model
anova(gls.rng.constvar, gls.rng)
```

Model	df	AIC	BIC	logLik	L.Ratio	p-value
Const Var	21	-184323.8	-184157.9	92182.92		
Our Model	40	-200413.0	-200096.9	100246.50	16127.15	< .0001

Table D.4.2: Comparison with Model assuming Constant Variance

This tests normality for residuals in each group.

```
# set an empty list
gls.groups.normality <- list(group=c(),p=c(),sig=c())

for (g in levels(groups)) {
  r <- res.rng[groups == g]

  # append data for current group
  gls.groups.normality$group <- c(gls.groups.normality$group, g)
  p.val <- shapiro.test(r)$'p.value'
  gls.groups.normality$p <- c(gls.groups.normality$p, p.val)

  # record significance
  if(p.val <= 0.05){
    gls.groups.normality$sig <- c(gls.groups.normality$sig, 'Not Normal')
  } else {
    gls.groups.normality$sig <- c(gls.groups.normality$sig, 'Normal')
  }
}

# saving text
gls.groups.normality <- as.data.frame(gls.groups.normality)
write.csv(gls.groups.normality, file='gls_group_residual_normality.csv', row.names = FALSE)
```

Outputs [D.4.2](#).

group	p	sig
LCG.300	8.62597709275305E-29	Not Normal
MT.300	1.22060741577847E-20	Not Normal
RANDU.300	2.63753155021491E-20	Not Normal
XOR.300	1.75427517417392E-21	Not Normal
LCG.600	4.33119837400064E-31	Not Normal
MT.600	0.00135805337582148	Not Normal
RANDU.600	0.00256271436401375	Not Normal
XOR.600	6.03526821037878E-06	Not Normal
LCG.900	5.82167414706182E-15	Not Normal
MT.900	0.0376199910094449	Not Normal
RANDU.900	0.929065688235714	Normal
XOR.900	0.00183537352944561	Not Normal
LCG.1200	1.98957127358888E-28	Not Normal
MT.1200	0.626128265476949	Normal
RANDU.1200	0.385643996191521	Normal
XOR.1200	0.241535365656695	Normal
LCG.1500	4.93777571502933E-23	Not Normal
MT.1500	0.111978683909765	Normal
RANDU.1500	0.00980778877854111	Not Normal
XOR.1500	0.351741238088697	Normal

Figure D.4.2: Residual Normality by Groups

Appendix E

Application: The Monte Carlo Estimation

E.1 Logics

Random sequences play a pivotal role in the Monte Carlo estimation [5]. The idea is based on the law of large numbers,

Theorem E.1 (The Law of Large Numbers). *Suppose $X_1, \dots, X_n$ with mean μ and variance σ^2 are independently sampled. Define $\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$ to be the sample mean. Then*

$$\frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \stackrel{n \rightarrow \infty}{\sim} N(0, 1) \quad (\text{E.1})$$

That is, $E(\bar{X})$ converges to μ as n gets larger.

Inspired by E.1, if we independently sample X sufficiently many times, the sample average $\bar{X}$ is an accurate estimate of $E(X)$. We can efficiently sample X using the inversion method.

Theorem E.2 (The Inversion Method). *Let X be a random variable with continuous and invertible cumulative distribution function $F(x) = P(X \leq x)$. Then*

$$F(X) \sim U(0, 1) \quad (\star)$$

and by invariant property of F ,

$$X \sim F^{-1}(U(0, 1)) \quad (\diamond)$$

Proof. Let $u \in [0, 1]$. We have

$$\begin{aligned} P(F(X) \leq u) &= P(F^{-1}(F(X)) \leq F^{-1}(u)) \text{ since } F \text{ is monotone} \\ &= P(X \leq F^{-1}(u)) \\ &= F(F^{-1}(u)) \\ &= u \text{ on } [0, 1] \end{aligned}$$

which is the CDF of $U(0, 1)$. So $F(X) \sim U(0, 1)$. The second equation can be proved in a similar way. $\square$

Suppose we have an independently sampled sequence of $U(0, 1)$,

$$U_1, U_2, \dots, U_n \stackrel{iid}{\sim} U(0, 1)$$

Then E.2 tells us that if $X \sim F$, an independent sample of X is obtained from

$$X_1 = F^{-1}(U_1), X_2 = F^{-1}(U_2), \dots, X_n = F^{-1}(U_n) \stackrel{iid}{\sim} X$$

The pseudo-random number generators are powerful tools to sample $U(0, 1)$. Numbers in the sequence follow $U(a, b)$ if they are chosen from $[a, b]$ where $a, b \in \mathbb{Z}, a < b$. By property,

$$U \sim U(a, b) \implies \frac{U - a}{b - a} \sim U(0, 1) \quad (\text{E.2})$$

In our experiment, the generated pseudo-random sequence

$$x_1, x_2, \dots, x_n \stackrel{iid}{\sim} U(1, 1024)$$

can be mapped to

$$\frac{x_1 - 1}{1023}, \frac{x_2 - 1}{1023}, \dots, \frac{x_n - 1}{1023} \stackrel{iid}{\sim} U(0, 1)$$

which is a sample of independent $U(0, 1)$ variables.

E.2 Estimating π

As a classic example of the Monte Carlo estimator, random sequences can be used to estimate π . Suppose

$$X_1, \dots, X_n, Y_1, \dots, Y_n \stackrel{iid}{\sim} U(0, 1)$$

are independently sampled. Define

$$D_i = (X_i, Y_i), \quad i \in \{1, \dots, n\}$$

to be a dot on the plane $(0, 0) \leq (x, y) \leq (1, 1)$, as shown in Figure (E.2.1).

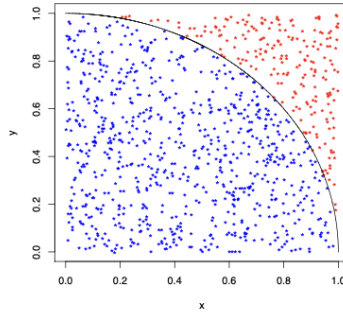


Figure E.2.1: Estimating π using MC

If X, Y 's are uniformly distributed, the fraction of D_i 's with norm less than or equal to 1 (the blue dots) should be around the area of the quarter-circle, $\frac{1}{4}\pi$, based on a sufficiently large sample size n . Therefore, an unbiased estimator of π such that $E(\hat{\pi}) = \pi$ is

$$\hat{\pi} = \frac{4}{n} \sum_{i=1}^n \mathbb{I} \{ \|D_i\| \leq 1 \} \quad (\text{E.3})$$

where $\mathbb{I} \{ \cdot \}$ is the indicator function.

Bibliography

- [1] Vivien Challis. *Pseudo-Random Number Generators*. URL: <https://www.maths.uq.edu.au/mathsed/phd/viviannotes.pdf>.
- [2] Elsevier. *Mersenne Twister*. (n.d.) URL: <https://www.sciencedirect.com/topics/computer-science/merse-ntwister>.
- [3] PIERRE L'ECUYER FRANÇOIS PANNETON. *On the Xorshift Random Number Generators*. 2004. URL: <https://www.iro.umontreal.ca/~lecuyer/myftp/papers/xorshift.pdf>.
- [4] Robert M. Gray. *Entropy and Information Theory*. (2023). URL: <https://ee.stanford.edu/~gray/it.pdf>.
- [5] IBM. *What is Monte Carlo Simulation?* (n.d.) URL: <https://www.ibm.com/think/topics/monte-carlo-simulation>.
- [6] Howard Rudd. *Hull and Dobell's first theorem*. (2023). URL: <https://www.howardrudd.net/mathematics/hull-and-dobells-first-theorem/>.
- [7] Mingheng S. *STAT 430 Project*. (2026). URL: <https://github.com/henry4516/STAT-430-Project.git>.